

Trabajo Práctico Grupal

Sistema de Recomendación con PySpark

Big Data Analytics con Spark · Licenciatura en Data Science

Contexto

En clase vimos cómo construir un **Two-Tower Recommender System** usando PySpark para el preprocesamiento y PyTorch para el modelo. Ahora les toca llevarlo a un problema real, con un dataset de su elección. No están obligados a usar el Two-Tower: más adelante van a encontrar tres modelos alternativos que pueden implementar en su lugar.

Hay una restricción de infraestructura: **tienen que ejecutarlo en Google Colab gratuito o en Databricks Community Edition**. Eso no es una limitación caprichosa — es el desafío real de tener que demostrar una idea antes de que les den infraestructura. Saber trabajar con lo que hay, y explicar cómo escalaría, es tan valioso como el modelo mismo.

Grupos de 3 a 4 personas.

Entornos disponibles

	Google Colab gratuito	Databricks Community Edition
RAM	~12 GB	~15 GB
GPU	T4 (no garantizada)	Sin GPU
Storage	Efímero (se pierde al cerrar)	DBFS persistente entre sesiones
PySpark	Se instala con apt + pip	Nativo, ya configurado
Límite práctico	toPandas() hasta ~500k filas	Similar, con ventaja de persistencia

Consignas

1. Elección del dataset y estrategia de ejecución

Elijan un dataset de dominio libre. En la sección de datasets al final hay cinco opciones, pero pueden proponer otro si lo justifican. El dataset va a ser más grande de lo que pueden procesar completo — eso es intencional. La consigna no es 'procesen todo', es 'planteen cómo trabajar con lo que tienen'.

Como parte de esta consigna tienen que responder brevemente:

- ¿Qué dataset eligieron y por qué?
- ¿Cuántos usuarios, ítems e interacciones tiene completo?
- ¿Qué subconjunto van a usar y cómo lo seleccionaron? (no vale agarrar los primeros N sin pensar)
- ¿En qué entorno van a correr y cómo manejan la persistencia entre sesiones?

2. Exploración y análisis con PySpark

Usen PySpark para explorar el dataset antes de tocar el modelo. Tienen que reportar:

- Distribución de ratings o interacciones
- Densidad de la matriz usuario-ítem (¿qué tan sparse es?)
- Distribución de interacciones por usuario y por ítem
- Al menos una observación no obvia que hayan descubierto en los datos

3. Estrategia de ejecución con recursos limitados

Tienen que documentar cómo adaptan el pipeline al entorno. No pueden simplemente correr el notebook de clase con otro dataset. Piensen en:

- Qué pasos son los más costosos en memoria y cómo los mitigan (por ejemplo: el crossJoin para generar negativos explota con datasets grandes, ¿qué alternativa usan?)
- Cómo evitan cargar todo en memoria a la vez
- Cómo configuran la SparkSession para el entorno (memoria del driver, particiones de shuffle)
- Qué sesgos puede introducir su estrategia de muestreo y qué métricas son confiables con su subconjunto

4. Preprocesamiento con PySpark

Todo el preprocesamiento tiene que correr en Spark. Como mínimo:

- Generación de ejemplos positivos y negativos, justificando la estrategia
- Re-indexación con StringIndexer
- Al menos dos features adicionales de usuario y dos de ítem (excepto ALS, ver consigna 5)
- Features normalizadas o codificadas correctamente, justificando las decisiones

5. Elección del modelo y entrenamiento

Pueden implementar cualquiera de estos cuatro modelos. Todos comparten el mismo pipeline de Spark. Lo que cambia es el modelo en sí. Tienen que justificar su elección.

Opción A — Two-Tower (el visto en clase)

Dos redes neuronales independientes que aprenden embeddings de usuarios e ítems y los comparan por producto punto. La opción más expresiva y la que mejor soporta features adicionales.

Requerimientos específicos:

- Cada torre combina embedding de ID con las features adicionales
- Justifican las dimensiones del embedding y del MLP
- La función de pérdida es apropiada para el tipo de dato

Opción B — ALS (Alternating Least Squares)

Nativo en PySpark (pyspark.ml.recommendation.ALS), corre 100% en Spark sin tocar PyTorch. Factoriza la matriz usuario-ítem en embeddings latentes. La opción más simple de implementar y la más fácil de escalar.

```
from pyspark.ml.recommendation import ALS als = ALS(maxIter=10, regParam=0.1,
userCol='user_index', itemCol='item_index', ratingCol='rating',
coldStartStrategy='drop') model = als.fit(train_df)
```

Requerimientos específicos:

- Experimentan con al menos dos valores de rank y regParam y reportan cuál da mejores métricas
- Discuten la limitación de no soportar features adicionales y cómo la resolverían

Opción C — BPR (Bayesian Personalized Ranking)

Aprende a ordenar en lugar de predecir ratings: el modelo rankea un ítem positivo por encima de uno negativo. Se implementa en PyTorch con embeddings simples y una loss basada en la diferencia de scores. Puente conceptual entre ALS y Two-Tower.

```
# El ítem positivo tiene que tener mayor score que el negativo diff = score_pos - score_neg loss = -torch.log(torch.sigmoid(diff)).mean()
```

Requerimientos específicos:

- El DataLoader genera tripletas (usuario, ítem positivo, ítem negativo)
- Justifican la estrategia de negative sampling en el training loop

Opción D — SVD++ simplificado

Extensión de ALS que incorpora el historial implícito del usuario: el hecho de haber interactuado con un ítem ya es señal. Se implementa en PyTorch agregando el promedio de embeddings del historial al embedding del usuario.

```
# Señal implícita: promedio de embeddings del historial implicit = item_embeddings[user_history].mean(dim=0) user_repr = user_embedding + implicit score = (user_repr * item_embedding).sum()
```

Requerimientos específicos:

- Construyen en Spark el historial de ítems por usuario antes de pasar a PyTorch
- Justifican cómo manejan usuarios con historial muy largo

Para todos los modelos, sin excepción: graficar curvas de entrenamiento, reportar al menos dos métricas de evaluación (*Precision@K*, *Recall@K* o *NDCG@K*), y mostrar recomendaciones concretas para al menos 3 usuarios discutiendo si tienen sentido.

6. Sección de escalabilidad

Esta sección no necesita código, necesita razonamiento. Describan qué cambiaría en el pipeline si tuvieran un cluster Databricks con 4 workers y GPU, 100 millones de interacciones, y la necesidad de actualizar las recomendaciones cada 24 horas. Hablen específicamente de:

- Cómo cambiaría el negative sampling a esa escala
- Por qué `toPandas()` deja de funcionar y qué lo reemplaza
- Cómo usarían `TorchDistributor` para el entrenamiento distribuido
- Cómo servirían las recomendaciones en tiempo real

Requerimientos mínimos para aprobar

Si falta alguno de estos ítems, el TP no aprueba independientemente del resto.

✓ El preprocesamiento corre íntegramente en PySpark

Ni pandas ni código Python puro para transformaciones

✓ Se justifica el subconjunto elegido

Criterio de muestreo documentado, no los primeros N registros

✓ Existe una sección de escalabilidad

Qué cambiaría con más recursos — sin código, con razonamiento

✓ **Se reportan métricas de evaluación sobre el conjunto de test**

Al menos dos: Precision@K, Recall@K o NDCG@K

✓ **El notebook corre de punta a punta sin errores desde cero**

Reiniciar kernel y ejecutar todo tiene que funcionar

✓ **El dataset es diferente a MovieLens 100k**

El del notebook de clase no cuenta

✓ **Se justifica la elección del modelo**

Por qué ese y no otro, considerando el dataset y el entorno

Lo que suma para una nota más alta

No es obligatorio, pero diferencia los TPs buenos de los muy buenos:

- + Comparación empírica de dos modelos de la lista sobre el mismo dataset y subconjunto
- + Comparación de dos estrategias de negative sampling y su impacto en las métricas
- + Experimento de ablation: modelo con features vs sin features
- + Visualización de embeddings con PCA o TSNE con interpretación
- + Uso de Parquet para persistir datos procesados entre sesiones (especialmente valioso en Databricks)
- + Benchmark de tiempo de las etapas principales del pipeline de Spark

Entrega

Archivo	Contenido
notebook.ipynb	Ejecutado con outputs visibles (o link al notebook de Databricks exportado)
README.md	Instrucciones para reproducir: qué entorno usar y cómo descargar el dataset
informe.pdf	Metodología, justificación del subconjunto y sección de escalabilidad (máx. 6 páginas)

El repositorio tiene que estar en GitHub con el README completo antes de la fecha límite.

Datasets recomendados

Cinco opciones de acceso libre, bien documentadas y con desafíos distintos. Para cada una se indica el subconjunto sugerido para trabajar en los entornos disponibles.

1

Amazon Product Reviews

Dominio: E-commerce | **Tamaño completo:** Millones de reseñas (varía por categoría)

URL: huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023

Subconjunto sugerido: Una categoría (Books o Electronics) filtrada a usuarios con 5-50 reseñas (~200-500k interacciones)

Por qué es bueno: Features ricas de producto (categoría, precio), ratings explícitos 1-5, posibilidad de usar embeddings de texto del título

Desafío de escala: La categoría Books completa tiene ~50M reseñas — pensar la estrategia de subconjunto es no trivial

2

Last.fm (HetRec 2011)

Dominio: Música | **Tamaño completo:** ~360k usuarios, 17M scrobbles

URL: grouplens.org/datasets/hetrec-2011

Subconjunto sugerido: Usuarios con entre 50 y 500 escuchas (~100k usuarios, ~5M interacciones)

Por qué es bueno: Interacciones implícitas (conteo de escuchas, no ratings), tags de artista como feature

Desafío de escala: El crossJoin para negativos sobre 360k usuarios × artistas revienta el entorno — el negative sampling eficiente es parte central del TP

3

Yelp Dataset

Dominio: Gastronomía y servicios | **Tamaño completo:** ~7M reseñas, 150k negocios

URL: [yelp.com/dataset](https://www.yelp.com/dataset)

Subconjunto sugerido: Una ciudad filtrada a usuarios con al menos 10 reseñas (~500k interacciones en ciudades grandes)

Por qué es bueno: Ratings explícitos 1-5, categorías detalladas, ubicación geográfica como feature potencial

Desafío de escala: El dataset completo no entra sin estrategia — obliga a filtrar inteligentemente desde Spark antes de cualquier operación costosa

4

Book-Crossing

Dominio: Libros | **Tamaño completo:** ~270k usuarios, 1M ratings

URL: [kaggle.com/datasets/syedjaferk/book-crossing-dataset](https://www.kaggle.com/datasets/syedjaferk/book-crossing-dataset)

Subconjunto sugerido: Se puede trabajar completo en Colab — el desafío está en el ruido, no en el tamaño

Por qué es bueno: El más accesible computacionalmente pero el más desafiante en calidad: tiene ratings implícitos y explícitos mezclados

Desafío de escala: Aunque es chico, es buen caso para practicar la arquitectura completa y proponer cómo escalaría

5

MovieLens 25M

Dominio: Películas | **Tamaño completo:** 25M ratings, 162k usuarios, 62k películas

URL: grouplens.org/datasets/movielens/25m

Subconjunto sugerido: Usuarios con 20-200 ratings y películas con al menos 50 ratings (~5M interacciones)

Por qué es bueno: Mismo dominio que el notebook de clase — permite comparar directamente. Géneros, año y tags como features

Desafío de escala: Pasar de 100k a 25M ratings no es solo 'más datos': los tiempos de shuffle se disparan y el negative sampling cambia

No elijan el dataset más fácil. Elijan el que más preguntas les genere. Los mejores trabajos siempre vienen de grupos que eligieron un problema que genuinamente les resultó interesante.